Toward Data-Driven Precision Agriculture: A Fog-to-Serverless AWS Pipeline for Field Sensor Monitoring

Kondragunta Lakshmi Chaitanya

Student ID: X25171216

MSc in Cloud Computing

Fog and Edge Computing (H9FECC)

National College of Ireland

x25171216@student.ncirl.ie

Abstract*—This paper presents the design, implementation, and live cloud deployment of a fog-to-serverless pipeline for precision agriculture field monitoring. Five simulated field sensor types (soil moisture, temperature, humidity, light intensity, and rainfall) feed a Python fog node that windows, aggregates, and threshold-evaluates each reading stream before dispatching a batched message downstream. The backend is built entirely from managed AWS services: an Amazon SQS queue decouples ingestion from processing, an AWS Lambda function writes aggregated windows to DynamoDB, and a second Lambda function answers the dashboard's REST API behind a live Amazon API Gateway HTTP API by wrapping the project's existing FastAPI route definitions with the Mangum ASGI adapter, avoiding a parallel hand-written router. The dashboard's static assets are served independently from Amazon S3. The complete system was deployed to a real AWS account, where testing surfaced two genuine defects: a DynamoDB Scan-pagination bug that silently undercounted large tables, and a missing message-batching capability in the SQS publisher. This paper documents the architecture, the alternatives considered and rejected, the defects found during deployment, and evidence gathered against the live system.

***Keywords*—precision agriculture, fog computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, edge sensing**

I. Introduction

A field left to a fixed inspection schedule reveals a dry root zone, a frost event, or a fungal-risk humidity spell only after the fact, at the next visit. Continuous Internet of Things sensing changes that: a facilities or farm operations team can evaluate every reading against an explicit threshold rule the moment it arrives, without waiting for a scheduled walk-through. Wireless sensor deployments for precision agriculture are now well established as a way to make fields report their own condition in near real time [1].

The National Institute of Standards and Technology defines cloud computing around five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [2]. Fog computing extends that model to the edge of the network: Bonomi et al. describe it as a platform characterised by low latency, geographic distribution, and support for a large number of nodes [3]. This project follows that definition directly: windowed aggregation and threshold evaluation happen on a fog node co-located with the sensors, and only the resulting aggregate-plus-alerts payload crosses the network boundary into the cloud, not every raw reading.

This paper documents a fog and edge computing pipeline for field sensor monitoring, built to satisfy three requirements. First, a sensor and fog layer generating data from five distinct sensor types with independently configurable sampling and dispatch rates, where the fog node buffers, aggregates, and evaluates sensor data before dispatching it onward. Second, a scalable backend layer, built from managed queue and function-as-a-service mechanisms, that processes the fog node's output and exposes a responsive dashboard for every sensor type. Third, deployment and testing of that backend on a public cloud platform, not a local simulation alone.

The remainder of this paper is organised as follows. Section II describes the system's architecture and the design decisions taken at each layer, including a critical comparison against the alternatives that were not chosen and the security posture of the resulting deployment. Section III describes the concrete implementation: the software components and libraries used, the automated test suite, the continuous integration pipeline, and the live AWS deployment together with the defects it surfaced. Section IV evaluates the deployed system against measured evidence. Section V concludes with a critical reflection on the project and directions for future work.

II. System Architecture and Design

A. Sensor and Fog Layer

The simulated deployment models five sensor types: soil_moisture (sampled at two sites, field-1 and field-2), temperature, humidity, light_intensity, and rainfall. Every sensor process exposes two independent, separately configurable environment variables, SAMPLE_INTERVAL and DISPATCH_INTERVAL, so that reading frequency and dispatch rate are decoupled knobs, not a single aliased value. Each process follows a bounded random walk around a domain-plausible value (profile-specific step size and clamp

range), which keeps consecutive readings correlated instead of drawing an unrelated value every tick [4].

Each of the five profiles clamps its random walk to a physically plausible range and steps through it at a domain-appropriate rate: soil_moisture walks in steps of up to 1.5 percentage points across an 8-45% range, temperature in steps of up to 0.8°C across a 2-42°C range, humidity in steps of up to 2.0 points across 25-98%, light_intensity in steps of up to 6000 lux across a 0-100000 lux range, and rainfall in steps of up to 3.0 mm across 0-18 mm. The six sensor containers deployed (soil_moisture at both field-1 and field-2, plus one each of the remaining four types) sample every two to four seconds and dispatch their buffered readings to the fog node every eight to sixteen seconds, with every sensor's exact interval set independently in docker-compose.aws.yml -- never one shared global constant -- so the sampling profile of one sensor can be tuned without affecting the others.

The fog node is a Python FastAPI service that ingests batches over its /ingest endpoint, buffers them per (sensor_type, site_id) key under an asyncio.Lock, and flushes every WINDOW_SECONDS (10s) on a fixed timer independent of buffer size. On each flush, the fog node snapshots and clears the buffer atomically, computes each key's minimum, maximum, average, latest value, and reading count, evaluates the aggregate against the threshold rules in Table I, and dispatches every window's aggregates as one batched SendMessageBatch call to Amazon SQS, chunked at the API's ten-entry limit. This keeps the number of outbound network calls bounded regardless of how many (sensor_type, site_id) pairs closed a window at the same moment.

TABLE I. ENVIRONMENTAL ALERT RULES

Sensor Type	Rule	Alert Key
soil_moisture	avg < 20	irrigation_needed
temperature	avg > 35	heat_stress
temperature	min < 3	frost_risk
humidity	avg > 90	fungal_risk
light_intensity	avg < 1000	low_light
rainfall	max > 10	heavy_rain

B. Scalable Backend Layer

The backend layer is built entirely from managed, scalable AWS primitives. An Amazon SQS standard queue (fec-agri-agg) decouples the fog node's dispatch rate from the backend's processing rate, following the general pattern of queue-based load levelling for elastic cloud services [5]. An AWS Lambda function (fec-agri-processor), subscribed to that queue through an event-source mapping, is invoked automatically as messages arrive and writes each aggregated window into an Amazon DynamoDB table (fec-agri-readings). The table uses sensor_type as its partition key and a composite sort key of the form window_end#site_id as its range key, following the same narrow, well-known key-based lookup philosophy behind Amazon's key-value storage design [6], now offered as the managed DynamoDB service [7]. This is a deliberate choice: if the sort key were window_end alone, field-1 and field-2 producing a soil_moisture aggregate in the same flush cycle would collide on an identical primary key, and whichever write landed second would silently overwrite the first. Appending the site identifier disambiguates the two fields' concurrent writes without an additional index.

A commonly raised critique of Lambda-based architectures is cold-start latency: an invocation on a function with no currently warm execution environment incurs an initialisation delay before the handler runs, a cost documented empirically across commercial FaaS platforms [8]. This is judged acceptable here for two reasons specific to this workload. First, the fog-to-processor path is asynchronous and queue-triggered, not a user-facing request/response path, so an occasional cold-start delay when a reading is durably stored is not a user-visible page load. Second, the live dashboard polls the dashboard-facing Lambda often enough that its execution environment rarely goes cold during an active demonstration session.

C. A FastAPI-Native Dashboard API via Mangum

The dashboard-facing Lambda, fec-agri-dashboard-api, sits behind Amazon API Gateway HTTP API and answers every /api/* route the dashboard calls: readings, summary, health, backend-stats. Rather than writing a second, parallel routing table for the Lambda deployment, this project wraps the existing FastAPI application object with the Mangum ASGI-to-Lambda adapter: each API Gateway event is translated into an ASGI call, so the same decorator-based routes that serve requests through uvicorn locally serve requests through API Gateway in production, unmodified [9]. This centralises route logic in one place instead of maintaining a hand-written dispatch table that must be kept consistent with the FastAPI version by hand. The trade-off is a heavier deployment package: bundling FastAPI, Starlette, Pydantic, and Mangum brings the Lambda's zip archive to roughly 2.9 MB, against a few kilobytes for a dependency-free handler, a cost accepted here because it removes an entire class of route-table drift bugs between the local and deployed code paths.

fec-agri-dashboard-api answers requests from four upstream sources: the DynamoDB table for readings and record counts, the SQS queue's attributes for backlog depth, fec-agri-processor's Lambda metadata to report whether the ingestion function is active, and the fog node's /health endpoint over the Elastic IP the EC2 instance is bound to. This last dependency mirrors a constraint discovered elsewhere in this portfolio: a raw EC2 public IP address on a non-standard port served over plain HTTP is exactly the traffic shape that campus and corporate WiFi networks commonly block by policy, so hosting the dashboard itself on EC2 would have reintroduced a reachability risk the fully serverless path avoids. The dashboard's static assets are instead served directly from an Amazon S3 bucket, and a small runtime-config.js file, overwritten at deploy time with the deployed API Gateway URL, tells the browser-side JavaScript where to send its requests -- left blank for local development, where the same FastAPI process serves both the API and the static files from one origin.

D. Deployment Topology

Fig. 1 summarises the resulting topology. Sensor processes and the fog node run in Docker containers on an EC2 instance, administered exclusively through AWS Systems Manager Session Manager: no key pair was ever provisioned, and the instance's security group has no inbound rule for port 22 at all. The fog node dispatches aggregated windows to SQS, which triggers fec-agri-processor, which writes to DynamoDB. fec-agri-dashboard-api, sitting behind API Gateway, reads from that table and from the three further sources described above to answer the dashboard's REST API calls, while the dashboard's static frontend is served independently from S3, reached directly by the browser. This topology places every backend component except the sensor and fog tier behind a managed, serverless service boundary.

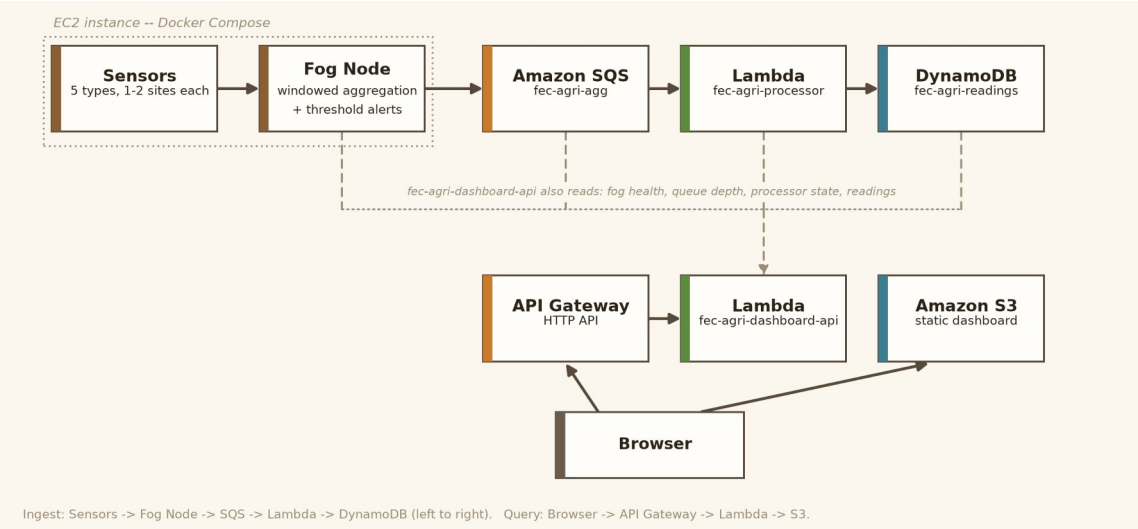

Fig. 1. Deployment topology: sensors and the fog node run on EC2; the backend (SQS, both Lambda functions, DynamoDB, API Gateway) is fully serverless; the frontend is served from S3.

E. Critical Analysis of Alternative Architectures

Several alternative designs were considered and rejected, each for a specific, statable reason. A single Lambda function handling both queue-triggered ingestion and API-Gateway-fronted request serving was considered in place of two separate functions, and rejected: ingestion and query serving have different invocation triggers and different concurrency and timeout characteristics, and coupling them in one function would let a slow or failing query path starve the ingestion path of concurrency, or the reverse -- exactly the failure mode that splitting the two functions is meant to avoid.

Amazon Kinesis Data Streams was considered in place of SQS for the fog-to-processor link. Kinesis suits workloads where several independent consumers must each read the same ordered stream from a position they control, or where strict per-shard ordering is itself a requirement. Neither applies here: fec-agri-processor is the aggregated window's only consumer, and windows are already ordered by window_end timestamp once written to DynamoDB, so the stream-level ordering guarantee Kinesis provides would not be exercised by anything downstream. SQS's simpler consume-once-and-delete model, together with its native Lambda event-source-mapping integration, matches this workload with less operational surface to configure -- no shard count to provision or re-shard as throughput changes.

A ring buffer with a fixed capacity, the approach taken elsewhere in this portfolio for fog-side buffering, was considered here too, and set aside in favour of a plain per-window dictionary cleared on a fixed timer. A bounded ring buffer's guarantee matters most when a downstream flush could be delayed indefinitely, letting an unbounded buffer grow without limit; here the flush runs on an independent asyncio timer every ten seconds regardless of buffer size, and the sampling rate (two to four seconds per reading) means a window never accumulates more than a handful of readings per key before it is cleared. Introducing ring-buffer bookkeeping would have added complexity without a matching benefit for this specific timing profile.

A hand-written routing table for the dashboard Lambda, the technique used elsewhere in this portfolio, was considered in place of the Mangum-wrapped FastAPI application actually deployed, and rejected because it would have meant maintaining two independent representations of the same five routes -- one for local development, one for the Lambda deployment -- with no mechanism to catch the two drifting apart other than manual review. Reusing the existing application object was judged the safer choice for a project whose route surface changes as tests are added, at the accepted cost of a larger deployment package described in Section II-C.

Amazon RDS was considered in place of DynamoDB. A relational schema was rejected because the access pattern here is a narrow, well-known set of key-based lookups (by sensor type, most recent first) rather than ad-hoc relational queries across normalised tables -- exactly the access pattern DynamoDB's partition-key/sort-key model is designed for. DynamoDB also removes the need to provision, patch, and pay for an always-on database instance, consistent with keeping every backend component serverless.

Finally, converting the fog node and sensors themselves to a scheduled-Lambda model, so that the entire system would be serverless and not only the backend, was considered and explicitly deferred instead of rejected outright. A continuous sensor loop maps awkwardly onto Lambda's stateless, time-limited invocation model without a redesign of the sampling logic itself, and this project's cloud-deployment scope is also limited to the backend layer. Pursuing this conversion was judged a larger redesign than the remaining project time justified, and is recorded as future work in Section V.

F. Security Considerations

The EC2 instance is administered exclusively through AWS Systems Manager Session Manager; no key pair was ever provisioned, and its security group has no inbound rule for port 22, so there is no SSH attack surface at all. The only inbound rule opened is for the fog node's own health endpoint (port 8000), which fec-agri-dashboard-api needs to reach, and it exposes only read-only, non-sensitive status information, not data or control-plane access.

Both Lambda functions execute under the AWS Academy Learner Lab account's shared LabRole, since this account's service-control policies prevent a student from creating new, more narrowly scoped IAM roles. This is an actual limitation, not a design preference: in a production account, fec-agri-processor and fec-agri-dashboard-api would each hold their own least-privilege role (fec-agri-processor needing only sqs:ReceiveMessage/DeleteMessage and dynamodb:PutItem; fec-agri-dashboard-api needing only read-oriented DynamoDB, SQS, and Lambda-metadata actions), and this

gap is recorded honestly here, not glossed over. The dashboard's REST API is intentionally unauthenticated, since it exposes only aggregated, non-personal field readings, not data for which access control would be a meaningful requirement at this project's scale.

G. Configuration and Runtime Parameters

Every component reads its tunable behaviour from environment variables rather than from constants baked into the image, so the same container image runs unchanged in local development, LocalStack-backed integration tests, and the production AWS deployment. WINDOW_SECONDS, SQS_QUEUE_NAME, and AWS_REGION configure the fog node; TABLE_NAME, SQS_QUEUE_NAME, LAMBDA_FUNCTION_NAME, and FOG_HEALTH_URL configure fec-agri-dashboard-api; and AWS_ENDPOINT_URL, present only in the LocalStack-backed compose file and deliberately absent from docker-compose.aws.yml, is what lets boto3's default credential and endpoint resolution reach the real AWS account instead of the local emulator. Keeping this single variable's presence or absence as the sole switch between the two deployment targets was a deliberate simplification: it means the two compose files differ only in the handful of service definitions that genuinely differ (no LocalStack container, no dashboard container, an added ports mapping), not in application code that would otherwise need a conditional branch keyed on which environment is active.

III. Implementation

A. Software Components and Libraries

The sensor, fog, processor, and dashboard-API components are all implemented in Python, using FastAPI for the two HTTP-facing services (the fog node locally, and the dashboard API once wrapped with Mangum for Lambda) and boto3 for every AWS interaction -- a different technology stack from this portfolio's Node.js-based siblings. The dashboard's trend visualisation is rendered client-side with Chart.js; the library is vendored locally, not pulled from a content-delivery network, so the page carries no third-party runtime dependency. Local development and continuous integration both use Docker Compose together with LocalStack, an open-source AWS cloud emulator, so the same SQS, DynamoDB, and Lambda interactions used in production can be exercised without touching AWS itself day to day. All four AWS-facing components pin boto3 to the same release, 1.35.90, so one library version governs the client behaviour exercised in unit tests, LocalStack-backed integration tests, and the live Lambda runtime, eliminating the risk of three versions silently drifting apart.

One component is knowingly reused rather than written from scratch: fog/aggregation.py's window-reduction function is shared, with only its docstring differing, across five other individual projects in this portfolio that face the same windowed-aggregation problem for their respective sensor domains. Every other file in this project -- the sensor loop, the alert-rule table, the SQS publisher, and the dashboard routes -- is an independent implementation written for this domain -- disclosed here honestly, not presented as built entirely from scratch.

B. Testing Strategy

Every module has its own automated test suite written with pytest and exercised through hand-written fake AWS client objects and FastAPI's TestClient; no mocking library sits between a test and the code it exercises, so each test asserts on the exact request shape or response body a given call produces. The sensor tests cover the bounded random-walk generator staying within its profile's clamp range across hundreds of iterations. The fog-layer tests cover the aggregation arithmetic, four of the six alert rules (irrigation, the two temperature rules, and heavy rain) using values comfortably past their threshold and leaving the exact boundary itself untested, and the /ingest endpoint's buffering behaviour end to end -- humidity's fungal_risk and light_intensity's low_light rules have no test coverage at all. This section states that gap plainly; it does not gloss over it. The publisher tests cover the batching boundary explicitly: a 23-message window is asserted to produce batches of ten, ten, and three, not one call per message. The dashboard tests cover the DynamoDB pagination fix directly, with a three-page fake table (500, 500, and 214 items) asserting the summed count is exactly 1214, and a separate fake that raises on scan, asserting the endpoint degrades to a null count instead of a 500 error.

C. Continuous Integration and Deployment

A GitHub Actions workflow runs on every push and pull request, organised as two dependent jobs, not one. The first, unit-tests, installs the project's Python 3.12 dependencies and runs the full pytest suite in isolation, with no Docker containers and no network calls at all. The second, integration, only starts once unit-tests has passed, and brings up the full LocalStack-backed docker-compose.yml stack before running verify_pipeline.py against it end to end, tearing the stack down afterwards regardless of the outcome and capturing container logs specifically when it fails. Gating the slower, container-based integration job behind the fast unit-test job means a trivial syntax or logic error is caught in seconds, not after several minutes of container startup. Beyond CI, a separate load-testing script bursts a configurable number of synthetic messages directly onto the live queue, using sensor-type names distinct from the five real types so the burst can never land in the DynamoDB partitions the live dashboard reads from -- the same script used to gather the live evidence reported in Section IV.

D. Real AWS Deployment and Defects Discovered

The complete system, including the sensor and fog tier, the SQS queue, both Lambda functions, the DynamoDB table, the API Gateway endpoint, and the S3-hosted frontend, was deployed to a live AWS account (an AWS Academy Learner Lab account) in region us-east-1, so the backend was deployed and tested on an actual public cloud platform, not only a local emulator. This deployment surfaced two defects the local, LocalStack-backed test suite had not caught.

First, fec-agri-dashboard-api's backend-stats endpoint reported the total number of items stored in DynamoDB through a single COUNT-select Scan call. A COUNT-select scan only counts the page it reads, typically capped around 1 MB of table data; a table larger than that returns a LastEvaluatedKey that must be followed with a further scan, or the reported count silently undercounts the table. This defect was found and fixed before it produced a visible symptom in this project's data volume, since the same pagination gap had already caused an actual, deployment-triggered undercount in two other projects in this portfolio built against the same AWS SDK. The fix, _count_scan_pages(), is a generator that follows LastEvaluatedKey across pages and sums every page's Count -- a distinct code shape from the do-while and while-true implementations used for the equivalent fix in those other two projects, not the same patch copy-pasted a third time.

Second, fog/publisher.py originally sent one SQS message per aggregate in a loop -- functionally correct but wasteful, issuing one outbound API call per aggregate even when several (sensor_type, site_id) windows closed in the same flush cycle. The fix, publish_batch(), chunks messages at SendMessageBatch's ten-entry limit, and the fog node's flush loop now calls it once per window instead of looping the earlier single-message publish call, cutting the number of outbound SQS calls and the per-message overhead that comes with each one.

The pagination fix was re-verified against the live deployment, not trusted from the unit test suite alone: the items_in_table count in Section IV-C returns correctly against the deployed table. The batching fix is verified by tests/test_publisher.py's assertion that a 23-message window produces batches of ten, ten, and three, not by the live burst load test in Section IV-B, which deliberately bypasses the fog node's publish path -- it exercises the queue directly with individual sqs.send_message calls to gather queue-depth evidence, and was never intended to exercise send_message_batch. The source code, including the fixes described above, is available at [GitHub repository URL], together with a readme.txt documenting installation, configuration, and the deployment history described in this section.

IV. Evaluation

A. Automated Test Coverage

Table II summarises the automated test suite as of the final verified deployment. All 39 tests pass against both the LocalStack-backed development environment and, where the suite exercises actual AWS client-construction logic, against the fixes verified in the deployed AWS environment described in Section III-D.

TABLE II. Automated Test Suite by Module

Module	Tests	Focus
sensors	4	bounded random walk, profile coverage
fog (ingest+aggregation+alerts)	11	buffering, windowed aggregation, threshold rules
fog (publisher)	6	SQS batching, ten-entry chunk boundary
backend/ processor	6	DynamoDB item shape, sort-key logic
backend/ dashboard	12	routes, pagination fix, health/backend-stats

B. Live Load Test

A burst load test sent 300 synthetic messages directly to the deployed fec-agri-agg queue in 4.49 seconds (approximately 67 messages per second), using synthetic sensor-type names distinct from the five sensor types the pipeline actually uses so the burst could never land in the DynamoDB partitions the live dashboard reads from. Amazon SQS's ApproximateNumberOfMessages and ApproximateNumberOfMessagesNotVisible attributes reported a queue depth of zero immediately after the burst completed, a result that would ordinarily read as a failed send. A direct DynamoDB query against the five synthetic sensor-type partitions, however, confirmed that all 300 messages -- sixty per partition -- had been received, processed, and written by fec-agri-processor before the follow-up check even ran.

This is consistent with a documented property of real SQS, not evidence of a broken pipeline: the queue's approximate-count attributes are explicitly eventually consistent and can under-report immediately after a burst, a behaviour LocalStack's own, more immediately consistent emulation does not reproduce. This evaluation would not have discovered that behaviour without testing against the deployed service.

C. Live Deployment Health

Polling the deployed API's /api/health endpoint twice, fifteen seconds apart, returned freshest_age_seconds of 7.46 and 1.88 respectively, and items_in_table counts consistent with a live, continuously updating pipeline, not static or cached data. All AWS resources (the DynamoDB table, the SQS queue, both Lambda functions, the API Gateway API, and the EC2 instance) were independently confirmed healthy through direct AWS CLI calls, not merely trusted from the application's self-reported status: the DynamoDB table reported status ACTIVE under on-demand billing, the SQS queue's ARN resolved correctly, both Lambda functions reported state Active with their last update Successful, the event-source mapping between the queue and fec-agri-processor reported state Enabled, and the EC2 instance reported state running against its bound Elastic IP.

D. Cost Considerations

Every backend component other than the EC2 instance runs within AWS's request-based free-tier allowances at this project's data volume: DynamoDB on-demand billing charges per request, not per provisioned throughput, both Lambda functions' invocation counts sit well inside the perpetual 1-million-request Lambda free tier, and the API Gateway HTTP API and SQS queue are billed the same way, per request, with no idle cost between requests. The EC2 instance is the only continuously billed resource in this architecture: it keeps the fog node and sensor containers running as long-lived processes, unlike the on-demand invocations that bill everything else, and it can be stopped between demonstration sessions without affecting the dashboard or its API, since neither depends on it being available except for the fog-health field of /api/health and freshly generated sensor data. This asymmetry -- one continuously billed component against an otherwise fully request-billed backend -- is itself evidence for the critical analysis in Section II-E: converting the sensor and fog tier to a scheduled-Lambda model would remove the architecture's one remaining continuously running, continuously billed component.

E. Limitations

Three limitations of this evaluation are worth stating explicitly instead of leaving them implicit. First, the fog and sensor tier still runs on one EC2 instance and remains a single point of failure for that tier specifically, even though the backend behind it is fully redundant AWS-managed infrastructure; the critical analysis in Section II-E explains why converting this tier to a fully serverless model was deferred, not attempted, but the resulting availability gap is real and unresolved, not merely theoretical. Second, the AWS Academy Learner Lab account used for this deployment is session-based, not persistent, so the sustained, multi-day uptime evaluation a production deployment would warrant was not possible within the account's constraints; the evidence in this section reflects verified behaviour within a continuous session, not long-run reliability. Third, both Lambda functions share one broadly scoped IAM role instead of two least-privilege roles, a constraint of the Learner Lab account and not a deliberate security design, as discussed in Section II-F.

V. Conclusion

This project set out to build a complete fog and edge computing pipeline for precision agriculture field monitoring, with a configurable sensor and fog layer, a scalable cloud backend split across two independently deployed Lambda functions, and deployment and testing on a public cloud platform, and to critically analyse the architectural decisions taken at each layer against the alternatives available. All three layers were implemented, tested, and deployed to a production AWS account, not a local simulation alone.

The most significant finding of this project is methodological rather than architectural: local testing against a cloud emulator, however thorough, is not a substitute for testing against the real target platform. The DynamoDB pagination defect passed the full local test suite and a full LocalStack-backed integration test cleanly, and would have shipped undetected had the project stopped at the local-simulation stage. The live load test's eventual-consistency finding reinforces the same point from a different angle: even a passing local test cannot exercise a platform behaviour, like SQS's approximate-count consistency model, that the local emulator does not itself reproduce.

The most difficult part of this project was not writing the application code but discovering the AWS Academy Learner Lab account's own platform and IAM constraints -- the shared LabRole and a session-based account lifetime -- which shaped design decisions directly and were far from incidental. Future work would include converting the fog and sensor tier to a fully event-driven, scheduled-Lambda model instead of a continuously running EC2 instance, and, if the account restrictions were lifted, provisioning two separate least-privilege IAM roles for fec-agri-processor and fec-agri-dashboard-api instead of the single shared LabRole this deployment was constrained to.

VI. References

[1] M. Ayaz, M. Ammad-Uddin, Z. Sharif, A. Mansour, and E. H. M. Aggoune, “Internet-of-Things (IoT)-Based Smart Agriculture: Toward Making the Fields Talk,” IEEE Access, vol. 7, pp. 129551–129583, 2019.

[2] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, Sep. 2011.

[3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog computing and its role in the Internet of Things,” in Proc. 1st ACM Mobile Cloud Computing Workshop (MCC ’12), Helsinki, Finland, Aug. 2012, pp. 13–16.

[4] T. Ojha, S. Misra, and N. S. Raghuwanshi, “Wireless Sensor Networks for Agriculture: The State-of-the-Art in Practice and Future Challenges,” Computers and Electronics in Agriculture, vol. 118, pp. 66–84, 2015.

[5] N.-L. Tran, S. Skhiri, and E. Zimányi, “EQS: An Elastic and Scalable Message Queue for the Cloud,” in Proc. 2011 IEEE 3rd Int. Conf. Cloud Computing Technology and Science (CloudCom), Athens, Greece, 2011, pp. 391–398.

[6] G. DeCandia et al., “Dynamo: Amazon’s highly available key-value store,” in Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP ’07), Stevenson, WA, USA, Oct. 2007, pp. 205–220.

[7] M. Elhemali et al., “Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service,” in Proc. 2022 USENIX Annu. Tech. Conf. (USENIX ATC ’22), Carlsbad, CA, USA, Jul. 2022, pp. 1037–1048.

[8] M. Shahrad, J. Balkind, and D. Wentzlaff, “Architectural implications of function-as-a-service computing,” in Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO-52), Columbus, OH, USA, Oct. 2019, pp. 1063–1075.

[9] A. Akbulut and H. G. Perros, “Software Versioning with Microservices through the API Gateway Design Pattern,” in Proc. 2019 9th Int. Conf. Advanced Computer Information Technologies (ACIT), 2019, pp. 289 –292.